Postman API Testing Guide

AI Math Solver & Tutor

Complete Backend API Reference

---

11 Public Endpoints | 10 Authenticated Endpoints
Django REST Framework | Gemini 2.5 Flash | JWT Authentication

Initial Setup

Base Configuration

Setting	Value
Base URL (Users)	http://127.0.0.1:8000/api/users/
Base URL (Chat)	http://127.0.0.1:8000/api/chat/
Default Header	Content-Type: application/json

Environment Variables (.env)

```
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
GEMINI_API_KEY=your_gemini_api_key
WOLFRAM_ALPHA_APP_ID=your_wolfram_app_id
GOOGLE_CLIENT_ID=your_google_client_id
SECRET_KEY=your_django_secret_key
```

Postman Environment Variables

Create two variables in Postman (Environments > Create New):

Variable	Description
access_token	Set after login — used in Authorization header
refresh_token	Set after login — used for token refresh

Before You Start

1. Run the Django server:

```
python manage.py runserver
```

2. Ensure migrations are up to date:

```
python manage.py makemigrations && python manage.py migrate
```

Phase 1: Registration Flow

Test 1.1 — Register a New User

Method	POST
URL	http://127.0.0.1:8000/api/users/register/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "username": "testuser",
    "password": "StrongPass123!",
    "password_confirm": "StrongPass123!"
}
```

Expected Response:

```
201 Created
{
    "message": "Registration successful. OTP sent to email."
}
```

Check your email/terminal for the 6-digit OTP. Save it for the next step.

Test 1.2 — Register with Mismatched Passwords (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/register/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser2@example.com",
    "username": "testuser2",
    "password": "StrongPass123!",
    "password_confirm": "DifferentPass456!"
}
```

Expected Response:

```
400 Bad Request
{
    "password": ["Passwords do not match."]
}
```

Test 1.3 — Register with Duplicate Email (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/register/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "username": "anothername",
    "password": "StrongPass123!",
    "password_confirm": "StrongPass123!"
}
```

Expected Response:

```
400 Bad Request
{
    "email": ["user with this email address already exists."]
}
```

Test 1.4 — Registration Rate Limit (Spam Protection)

Method	POST
URL	http://127.0.0.1:8000/api/users/register/

Expected Response:

```
429 Too Many Requests
{
    "detail": "Request was throttled. Expected available in X seconds."
}
```

Send 15+ registration requests within 1 hour. OTPRateThrottle blocks further attempts.

Phase 2: Email Verification

Test 2.1 — Verify Email with Correct OTP

Method	POST
URL	http://127.0.0.1:8000/api/users/verify-email/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "otp": "123456"
}
```

Expected Response:

```
200 OK
{
    "message": "Email verified successfully."
}
```

Replace 123456 with the actual OTP from your email/terminal.

Test 2.2 — Verify with Wrong OTP (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/verify-email/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "otp": "000000"
}
```

Expected Response:

```
400 Bad Request
{
    "error": "Invalid OTP."
}
```

Test 2.3 — Verify with Expired OTP (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/verify-email/

Expected Response:

```
400 Bad Request
{
    "error": "OTP expired."
}
```

Wait more than 10 minutes after registration. Use Resend OTP (Phase 3) to get a fresh one.

Phase 3: Resend OTP

Test 3.1 — Resend Verification OTP

Method	POST
URL	http://127.0.0.1:8000/api/users/resend-otp/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "purpose": "verification"
}
```

Expected Response:

```
200 OK
{
    "message": "OTP resent successfully. Please check your email."
}
```

Test 3.2 — Resend Too Quickly (Cooldown Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/resend-otp/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "purpose": "verification"
}
```

Expected Response:

```
429 Too Many Requests
{
    "error": "Please wait 55 seconds before requesting a new OTP.",
    "retry_after": 55
}
```

Send immediately after Test 3.1. The retry_after field tells the frontend how many seconds to display in the countdown.

Test 3.3 — Resend for Already Verified User (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/resend-otp/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "purpose": "verification"
}
```

Expected Response:

```
400 Bad Request
{
    "error": "Email is already verified."
}
```

Test 3.4 — Resend Password Reset OTP

Method	POST
URL	http://127.0.0.1:8000/api/users/resend-otp/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "purpose": "password_reset"
}
```

Expected Response:

```
200 OK
{
    "message": "OTP resent successfully. Please check your email."
}
```

Phase 4: Login

Test 4.1 — Login with Verified User

Method	POST
URL	http://127.0.0.1:8000/api/users/login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "username": "testuser",
    "password": "StrongPass123!"
}
```

Expected Response:

```
200 OK
{
    "message": "Login successful",
    "tokens": {
        "refresh": "eyJhbG...",
        "access": "eyJhbG..."
    },
    "user": {
        "id": 1,
        "username": "testuser",
        "email": "testuser@example.com"
    }
}
```

IMPORTANT: Copy the access and refresh tokens into your Postman environment variables.

Test 4.2 — Login with Unverified User (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/login/

Expected Response:

```
403 Forbidden
{
    "error": "Email not verified."
}
```

Register a user but don't verify their email, then try to login.

Test 4.3 — Login with Wrong Password (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "username": "testuser",
    "password": "WrongPassword!"
}
```

Expected Response:

```
401 Unauthorized
{
    "error": "Invalid credentials."
}
```

Test 4.4 — Login with Mismatched Username (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "username": "wrongusername",
    "password": "StrongPass123!"
}
```

Expected Response:

```
400 Bad Request
{
    "error": "Username does not match email."
}
```

Phase 5: Google Login

How to Get a Google id_token for Testing

Option A: Create a test_google.html file with Google Sign-In button, open in browser, copy the token.

Option B: Use Google OAuth 2.0 Playground (https://developers.google.com/oauthplayground/).

Test 5.1 — Google Login (New User — Auto-Created)

Method	POST
URL	http://127.0.0.1:8000/api/users/google-login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "id_token": "eyJhbGciOiJSUzI1NiIs..."
}
```

Expected Response:

```
200 OK
{
    "message": "Google Login successful",
    "tokens": {
        "refresh": "eyJhbG...",
        "access": "eyJhbG..."
    },
    "user": {
        "id": 2,
        "username": "johndoe8a3f2b1c",
        "email": "johndoe@gmail.com"
    }
}
```

New user auto-created with is_verified=True. No OTP needed. Username generated from email prefix + random suffix.

Test 5.2 — Google Login (Returning User)

Method	POST
URL	http://127.0.0.1:8000/api/users/google-login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "id_token": "eyJhbGciOiJSUzI1NiIs..."
}
```

Expected Response:

```
200 OK — same user.id as Test 5.1
```

Proves the backend finds existing user instead of creating a duplicate.

Test 5.3 — Google Login with Missing Token (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/google-login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
}
```

Expected Response:

```
400 Bad Request
{
    "error": "id_token is required"
}
```

Test 5.4 — Google Login with Invalid Token (Error)

Method	POST
URL	http://127.0.0.1:8000/api/users/google-login/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "id_token": "this.is.not.a.real.token"
}
```

Expected Response:

```
401 Unauthorized
{
    "error": "Invalid token: ..."
}
```

Phase 6: Token Refresh

Test 6.1 — Refresh Access Token

Method	POST
URL	http://127.0.0.1:8000/api/users/token/refresh/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "refresh": "{{refresh_token}}"
}
```

Expected Response:

```
200 OK
{
    "access": "eyJhbGciOiJIUzI1NiIs..."
}
```

Update your access_token environment variable with the new value.

Phase 7: Profile Management

Test 7.1 — Get Profile

Method	GET
URL	http://127.0.0.1:8000/api/users/profile/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
200 OK
{
    "id": 1,
    "username": "testuser",
    "email": "testuser@example.com"
}
```

Test 7.2 — Update Username

Method	PUT
URL	http://127.0.0.1:8000/api/users/profile/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "username": "updated_testuser"
}
```

Expected Response:

```
200 OK
{
    "id": 1,
    "username": "updated_testuser",
    "email": "testuser@example.com"
}
```

Test 7.3 — Get Profile Without Token (Error)

Method	GET
URL	http://127.0.0.1:8000/api/users/profile/
Headers	None

Expected Response:

```
401 Unauthorized
```

```
{
    "detail": "Authentication credentials were not provided."
}
```

Phase 8: Password Reset Flow (3 Steps)

Test 8.1 — Step 1: Request Password Reset

Method	POST
URL	http://127.0.0.1:8000/api/users/password-reset/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com"
}
```

Expected Response:

```
200 OK
{
    "message": "OTP sent to email."
}
```

Save the OTP from email/terminal. Also has 60-second cooldown per email.

Test 8.2 — Step 2: Verify Password Reset OTP

Method	POST
URL	http://127.0.0.1:8000/api/users/password-reset/verify/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "email": "testuser@example.com",
    "otp": "654321"
}
```

Expected Response:

```
200 OK
{
    "message": "OTP verified. Proceed to reset password."
}
```

This sets a session cookie. Ensure Postman cookies are enabled.

Test 8.3 — Step 3: Set New Password

Method	POST
URL	http://127.0.0.1:8000/api/users/password-reset/confirm/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
```

```
    "new_password": "NewStrongPass456!",
    "password_confirm": "NewStrongPass456!"
}
```

Expected Response:

```
200 OK
{
    "message": "Password reset successful."
}
```

Verify by logging in with the new password.

Phase 9: Chat Sessions (Authenticated)

Test 9.1 — Create a New Chat Session

Method	POST
URL	http://127.0.0.1:8000/api/chat/sessions/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
201 Created
{
    "id": 1,
    "user": 1,
    "title": null,
    "created_at": "2026-03-04T10:00:00Z",
    "updated_at": "2026-03-04T10:00:00Z",
    "messages": []
}
```

Save the id value — you will use it as {session_id} in subsequent tests.

Test 9.2 — List All Chat Sessions (Sidebar — Paginated)

Method	GET
URL	http://127.0.0.1:8000/api/chat/sessions/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
200 OK
{
    "count": 1,
    "next": null,
    "previous": null,
    "results": [
        {
            "id": 1,
            "title": null,
            "created_at": "...",
            "updated_at": "..."
        }
    ]
}
```

Paginated (max 20/page). Lightweight serializer — no messages, no user field.

Test 9.3 — Get Single Session Detail (With Messages)

Method	GET
URL	http://127.0.0.1:8000/api/chat/sessions/1/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
200 OK — full serializer with messages array included
```

Test 9.4 — Rename a Chat Session

Method	PUT
URL	http://127.0.0.1:8000/api/chat/sessions/1/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "title": "Quadratic Equations Practice"
}
```

Expected Response:

```
200 OK — updated title in response
```

Test 9.5 — Rename with Empty Title (Error)

Method	PUT
URL	http://127.0.0.1:8000/api/chat/sessions/1/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "title": "   "
}
```

Expected Response:

```
400 Bad Request
{
    "error": "Title is required and cannot be empty."
}
```

Phase 10: Sending Messages (Authenticated — History Saved)

Test 10.1 — Send a Text Message

Method	POST
URL	http://127.0.0.1:8000/api/chat/sessions/1/send/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "message": "Solve x^2 + 5x + 6 = 0"
}
```

Expected Response:

```
200 OK — returns both user_message and ai_response objects
```

Both messages saved to database. Session title auto-updates on first message.

Test 10.2 — Send with Language Hint

Method	POST
URL	http://127.0.0.1:8000/api/chat/sessions/1/send/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "message": "Solve x^2 + 5x + 6 = 0",
    "language": "Bulgarian"
}
```

Expected Response:

```
200 OK — AI responds in Bulgarian
```

Optional parameter. If omitted, Gemini auto-detects from the user's input language.

Test 10.3 — Send with Image

Body type: form-data (NOT raw JSON)

Key	Type	Value
message	Text	Help me solve this problem
image	File	(Select an image file)

Expected: 200 OK with full image URL in response. AI reads the image content.

Test 10.4 — Send with Audio

Body type: form-data

Key	Type	Value
message	Text	(optional)
audio	File	(Select .mp3 or .wav file)

Expected: 200 OK. Audio uploaded to Gemini, processed, then cleaned up to prevent quota leak.

Test 10.5 — Verify Context Awareness (Chat Memory)

Method	POST
URL	http://127.0.0.1:8000/api/chat/sessions/1/send/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
First:  { "message": "Let x = 5 and y = 10" }
Second: { "message": "What is x + y?" }
```

Expected Response:

```
AI should respond with 15 — proves chat history is loaded from database.
```

Phase 11: Guest Chat (No Auth — History NOT Saved)

Test 11.1 — Guest Text Chat

Method	POST
URL	http://127.0.0.1:8000/api/chat/guest/
Headers	Content-Type: application/json (NO Authorization header)

Body (raw JSON):

```
{
    "message": "What is the integral of x^2?"
}
```

Expected Response:

```
200 OK
{
    "message": "What is the integral of x^2?",
    "ai_response": "The integral of x² is (x³/3) + C...",
    "is_guest": true
}
```

No auth required. is_guest: true helps frontend show "Join to save your chats". Nothing saved to database.

Test 11.2 — Guest Chat with Language Hint

Method	POST
URL	http://127.0.0.1:8000/api/chat/guest/
Headers	Content-Type: application/json

Body (raw JSON):

```
{
    "message": "Реши x^2 + 5x + 6 = 0",
    "language": "Bulgarian"
}
```

Expected Response:

```
200 OK — AI responds in Bulgarian
```

Test 11.3 — Guest Chat with Image

Body type: form-data (NO Authorization header)

Key	Type	Value
message	Text	Solve this
image	File	(Select a math problem image)

Test 11.4 — Guest Chat with Audio

Body type: form-data (NO Authorization header)

Key	Type	Value
message	Text	(optional)
audio	File	(Select .mp3 or .wav file)

No file saved to disk. Audio cleaned up from Gemini servers after processing.

Test 11.5 — Guest Chat Has No Memory

Method	POST
URL	http://127.0.0.1:8000/api/chat/guest/
Headers	Content-Type: application/json

Body (raw JSON):

```
First:  { "message": "Let x = 100" }
Second: { "message": "What is x?" }
```

Expected Response:

```
AI should NOT know what x is — each request is independent.
```

Phase 12: Terms & Privacy

Test 12.1 — Get Terms and Privacy Policy

Method	GET
URL	http://127.0.0.1:8000/api/users/terms-privacy/

Expected Response:

```
200 OK
{
    "title": "Terms and Privacy Policy",
    "content": "This Privacy Policy describes how..."
}
```

Phase 13: Delete Session

Test 13.1 — Delete a Chat Session

Method	DELETE
URL	http://127.0.0.1:8000/api/chat/sessions/1/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
204 No Content
{
    "message": "Session deleted"
}
```

Verify: GET /api/chat/sessions/1/ should return 404.

Phase 14: Account Management

Test 14.1 — Delete Account

Method	DELETE
URL	http://127.0.0.1:8000/api/users/profile/
Headers	Authorization: Bearer {{access_token}}

Expected Response:

```
200 OK
{
    "message": "Account deleted successfully."
}
```

WARNING: Permanently deletes user and all chat sessions (CASCADE). Test this last!

Phase 15: Logout

Test 15.1 — Logout (Blacklist Refresh Token)

Method	POST
URL	http://127.0.0.1:8000/api/users/logout/
Headers	Authorization: Bearer {{access_token}}, Content-Type: application/json

Body (raw JSON):

```
{
    "refresh": "{{refresh_token}}"
}
```

Expected Response:

```
205 Reset Content
{
    "message": "Logout successful."
}
```

Verify: POST /api/users/token/refresh/ with the same token should fail.

Phase 16: Rate Limiting Verification

Send requests exceeding the limits below. Expected response: 429 Too Many Requests.

Scope	Limit	Applies To
Anonymous	30 requests/minute	All unauthenticated endpoints
Authenticated	60 requests/minute	All authenticated endpoints
OTP Endpoints	15 requests/hour per IP	Register, Password Reset, Resend OTP
OTP Cooldown	60 seconds per email	Resend OTP, Password Reset Request

Complete Endpoint Reference

Public Endpoints (No Auth)

Method	URL	Description
POST	/api/users/register/	Register new user (rate limited)
POST	/api/users/verify-email/	Verify email with OTP
POST	/api/users/resend-otp/	Resend OTP (rate limited, 60s cooldown)
POST	/api/users/login/	Login with email + username + password
POST	/api/users/google-login/	Login with Google id_token
POST	/api/users/token/refresh/	Refresh access token
POST	/api/users/password-reset/	Request password reset (rate limited)
POST	/api/users/password-reset/verify/	Verify reset OTP
POST	/api/users/password-reset/confirm/	Set new password
GET	/api/users/terms-privacy/	Get terms & privacy
POST	/api/chat/guest/	Guest chat (no history saved)

Authenticated Endpoints (Bearer Token Required)

Method	URL	Description
GET	/api/users/profile/	Get profile
PUT	/api/users/profile/	Update profile (username)
DELETE	/api/users/profile/	Delete account
POST	/api/users/logout/	Logout (blacklist token)
GET	/api/chat/sessions/	List sessions (paginated, lightweight)
POST	/api/chat/sessions/	Create new session
GET	/api/chat/sessions/{id}/	Get session with all messages
PUT	/api/chat/sessions/{id}/	Rename session
DELETE	/api/chat/sessions/{id}/	Delete session
POST	/api/chat/sessions/{id}/send/	Send message (text/image/audio)